TUM School of Computation, Information and Technology
Technical University of Munich

Semester Thesis

English Title

German Title

Supervisor Prof. Dr.-Ing. habil. Alois C. Knoll

Advisor Zhang Chi, M.Sc.

Author Carsten Oertel

Date April 15, 2025 in Munich

Disclaimer

I confirm that this Semester Thesis is my own work and I have documented all sources and material used.

Munich, April 15, 2025

(Carsten Oertel)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Kurzfassung

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Abbreviations **xi**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Contributions . . . 2
1.4 Thesis Outline . . . 2

2 Related Work **5**
2.1 Classical Robot Control . . . 5
2.2 Classical Grasp Planning and Dexterous Manipulation . . . 6
2.3 Reinforcement Learning . . . 7
2.4 Sim-to-Real Transfer . . . 8
2.5 Imitation Learning and Policy Distillation . . . 9
2.6 Dexterous Grasping and Manipulation . . . 10
2.7 Simulation Frameworks . . . 11

3 Methods **13**
3.1 Problem Formulation . . . 13
3.1.1 Dexterous Grasping as a POMDP . . . 13
3.1.2 Privileged vs. Partial Observability . . . 13
3.2 Privileged Teacher Training . . . 14
3.2.1 Asymmetric Actor–Critic . . . 14
3.2.2 PPO Objective . . . 14
3.2.3 Reward Structure . . . 15
3.2.4 Adaptive Domain Randomization . . . 15
3.3 Policy Distillation . . . 15
3.3.1 DAgger . . . 16
3.3.2 SafeDAgger . . . 16

4 Experiments and Results **19**
4.1 Experimental Setup . . . 19
4.1.1 Hardware Platform . . . 19
4.1.2 Simulation Environment . . . 20
4.1.3 Object Set . . . 21
4.1.4 Episode Structure . . . 21
4.2 Teacher Policy Implementation . . . 21
4.2.1 Observation Space . . . 21
4.2.2 Action Space . . . 21
4.2.3 Reward Design . . . 22
4.2.4 Network Architecture . . . 22

4.2.5 Training Configuration . . . 23
4.2.6 Adaptive Domain Randomization . . . 23
4.3 Student Policy Implementation . . . 23
4.3.1 Student Observation Space . . . 24
4.3.2 Student Network Architecture . . . 24
4.3.3 SafeDagger Distillation Configuration . . . 24
4.4 Evaluation Protocol . . . 24
4.4.1 Metrics . . . 24
4.4.2 Evaluation Procedure . . . 25
4.5 Teacher Training Results . . . 25
4.5.1 Training Progression . . . 25
4.5.2 Physics Stability Challenges . . . 25
4.5.3 Teacher Performance . . . 26
4.6 Distillation Results . . . 26
4.6.1 SafeDAgger Distillation . . . 27
4.6.2 DAgger Baseline Comparison . . . 28
4.6.3 Safety Analysis . . . 29
4.6.4 Intervention Analysis . . . 29
4.7 Ablation Studies . . . 29
4.7.1 Effect of Number of Training Environments . . . 29
4.7.2 Effect of Early Termination Penalty . . . 29
4.7.3 Effect of Teacher Strength on Distillation . . . 29
4.7.4 LSTM Checkpoint Resume for ADR Curriculum . . . 30
4.7.5 Safety Threshold Sensitivity . . . 30
4.7.6 Per-Object Performance Breakdown . . . 31
4.7.7 Query Efficiency Analysis . . . 31
4.7.8 Learning Curve Comparison . . . 32
4.7.9 Effect of Distillation Duration . . . 32
4.8 Sim-to-Real Curriculum . . . 32
4.8.1 Curriculum Design . . . 32
4.8.2 Preliminary Results . . . 33
4.9 Summary of Findings . . . 33

5 Discussion and Conclusion **35**

A Appendix 1 **37**

Bibliography **39**

List of Figures

2.1 Illustration of domain randomization. A single calibrated simulation (red circle) may not match reality (green triangle), but a sufficiently wide distribution of randomized simulations (dashed ellipses) can encompass the real-world conditions, enabling zero-shot transfer. Figure adapted from [23]. 8

List of Tables

4.1 PhysX simulation parameters. 20
4.2 Actuator parameters for the FR3 + Agile Hand system. 20
4.3 Teacher actor observation space. 22
4.4 Reward terms and weights for teacher training. 22
4.5 PPO hyperparameters for teacher training. 23
4.6 Adaptive Domain Randomization parameters. 23
4.7 Physics stability challenges encountered during teacher training. 26
4.8 Teacher policy performance (multi-object, 13 VisDex objects). The ADR-19 teacher was evaluated over 80 episodes (16 envs × 5 rollouts) at ADR-0 physics. 26
4.9 SafeDAgger distillation results (during-training metrics, including teacher-assisted actions). 27
4.10 Standalone student vs. teacher performance (80 episodes, ADR-0 physics). . . 27
4.11 Head-to-head comparison of DAgger and SafeDAgger distillation (ADR-19 teacher, 13 objects, 350k iterations, 80 standalone evaluation episodes each). 28
4.12 Effect of safety threshold δ on SafeDAgger distillation performance (ADR-19 teacher, 350k iterations, standalone evaluation over 80 episodes). 30
4.13 Per-object Grasp Success Rate (GSR) (%) for teacher, SafeDAgger student, and DAgger student (80 evaluation episodes per object, ADR-0 physics). 31

List of Abbreviations

RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
IL	Imitation Learning
BC	Behavioral Cloning
PPO	Proximal Policy Optimization
TRPO	Trust Region Policy Optimization
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
DR	Domain Randomization
ADR	Automatic Domain Randomization
DoF	Degree of Freedom
FR3	Franka Research 3
FGP	Fabric-Guided Policy
GWS	Grasp Wrench Space
GSR	Grasp Success Rate
UER	Unsafe Episode Rate
TQR	Teacher Query Rate
LSTM	Long Short-Term Memory
USD	Universal Scene Description

Chapter 1

Introduction

This chapter motivates the problem of dexterous robotic grasping, states the specific challenges addressed in this work, summarizes the contributions, and outlines the structure of this thesis.

1.1 Motivation

Multi-fingered dexterous hands offer robots the potential to interact with the world in a manner approaching human versatility. Unlike parallel-jaw grippers, which are limited to a narrow set of grasp geometries, anthropomorphic hands can conform to arbitrary object shapes, reorient objects in-hand, and wield tools [1]. Despite this potential, high Degree of Freedom (DoF) hands remain largely confined to research laboratories. The core bottleneck is not hardware, as several capable platforms exist, including the Allegro Hand, the Shadow Dexterous Hand, and the Agile Hand used in this work, but rather the difficulty of synthesizing reliable control policies for contact-rich tasks.

Traditional analytic approaches to grasp planning rely on known object models and precise contact modeling, assumptions that rarely hold in unstructured environments [2]. Learning-based methods, particularly Deep Reinforcement Learning (DRL), have recently demonstrated impressive dexterous manipulation capabilities in simulation [3, 4]. However, transferring these policies to real hardware introduces a fundamental challenge: during training, RL agents have access to privileged information (exact object poses, contact forces, full state vectors) that is unavailable at deployment time, where the robot must rely on noisy sensory observations such as camera images.

The dominant strategy to bridge this gap is *policy distillation*, in which a privileged *teacher* policy is trained in simulation with full state access and its behavior is then distilled into a *student* policy that operates from realistic sensor inputs [5, 6]. The DAgger algorithm [7] is widely used for this distillation step, as it addresses the distribution shift between expert demonstrations and student-induced trajectories through iterative data aggregation. Yet standard DAgger is agnostic to safety: it queries the teacher uniformly, including in states where the student already performs well, and does not prevent the student from visiting dangerous states during training. For dexterous hands, where unsafe actions can cause hardware damage (e.g., jamming or overloading finger joints) or lead to unstable contact configurations, a safety-aware distillation procedure is desirable.

1.2 Problem Statement

This thesis addresses the problem of learning a vision-based dexterous grasping policy through simulation, with a focus on making the distillation from a privileged teacher to a vision-based student both effective and safe.

Concretely, the work builds on the DextrAH-RGB framework [5], which proposes a two-stage pipeline:

1. A **privileged teacher policy** is trained via Reinforcement Learning (RL) (Proximal Policy Optimization (PPO)) in a GPU-accelerated physics simulator with access to ground-truth object poses, joint states, and contact information.

2. A **vision-based student policy** is distilled from the teacher using DAgger, learning to map RGB observations to grasp actions.

The original DextrAH-RGB work employs Fabric-Guided Policys (FGPs) built on geometric fabrics [8] and uses DAgger for distillation. This thesis departs from the original approach in two key ways:

- The teacher policy is trained as a standard RL policy **without geometric fabrics**, using custom reward functions in an Isaac Lab direct environment (`DirectRLEnv`) with the Agile Hand mounted on a Franka Research 3 (FR3) arm.

- The distillation step replaces DAgger with **SafeDAgger** [9], a query-efficient variant that learns a safety classifier to determine when the teacher should intervene, thereby reducing unsafe episodes during training. This is further informed by DexSafeDagger [10], an unpublished extension of SafeDAgger that incorporates Q-value-based failure prediction for dexterous manipulation.

1.3 Contributions

The main contributions of this semester thesis are:

1. **Dexterous grasping environment in Isaac Lab:** A direct RL environment (`DirectRLEnv`) in Isaac Lab for training arm-hand grasping policies with the Agile Hand and FR3, including custom reward shaping and asset integration.

2. **Privileged teacher policy:** A trained RL teacher policy that achieves robust dexterous grasping of diverse objects in simulation using privileged state information.

3. **SafeDAgger-based distillation:** A comparative study of DAgger versus SafeDAgger for distilling the privileged teacher into a vision-based student, evaluating both task performance (GSR) and training safety (number of unsafe episodes).

This work lays the foundation for the subsequent master's thesis, which will extend the pipeline to include finger-gaiting, in-hand reorientation, and fine tool use such as screwdriving.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

Chapter 2 reviews related work on dexterous grasping, policy distillation methods (DAgger, SafeDAgger), and sim-to-real transfer for manipulation.

Chapter 3 formalizes the problem as a Partially Observable Markov Decision Process (POMDP), derives the privileged teacher training objective, and presents the mathematical framework for DAgger, SafeDAgger, and DexSafeDagger distillation.

Chapter 4 describes the experimental setup, including the hardware platform, simulation environment, reward design, and network architectures, presents teacher training results and SafeDAgger distillation experiments, and compares DAgger and SafeDAgger.

Chapter 5 discusses the findings, limitations, and directions for future work in the master's thesis.

Chapter 2

Related Work

This chapter surveys the foundations and recent advances relevant to this thesis. It begins with classical robot control and grasp planning, discusses their limitations for dexterous manipulation, and motivates the shift to data-driven methods. It then covers reinforcement learning and sim-to-real transfer, followed by imitation learning and policy distillation, and concludes with the state of the art in dexterous grasping and the simulation frameworks that enable it.

2.1 Classical Robot Control

The control of robotic manipulators has a rich history rooted in classical control theory. A robot arm with n joints is described by its rigid-body dynamics [11]:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} \tag{2.1}$$

where $\boldsymbol{q} \in \mathbb{R}^n$ is the joint configuration, $\boldsymbol{M}$ the inertia matrix, $\boldsymbol{C}$ the Coriolis and centrifugal term, $\boldsymbol{g}$ the gravity vector, and $\boldsymbol{\tau}$ the applied joint torques.

Position control. The most common approach to manipulator control is computed torque control, which uses an inverse dynamics model to linearize the system, followed by a PD controller in joint space. For a desired trajectory $\boldsymbol{q}_d(t)$, the control law takes the form $\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})(\ddot{\boldsymbol{q}}_d + K_p\boldsymbol{e} + K_d\dot{\boldsymbol{e}}) + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$, where $\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$ is the tracking error [11]. PD position control is used extensively in both simulation and deployment for this work: in simulation, each joint is driven by a PD controller that tracks the policy's position targets, and on the real robot, the low-level firmware runs PD loops at 1 kHz. While effective for trajectory following, pure position control assumes a known dynamics model and does not explicitly account for interaction forces, which can lead to excessive contact forces during grasping.

Joint-space impedance control. When a robot must interact with objects or the environment, pure position control risks generating dangerously high forces upon contact. Hogan [12] introduced *impedance control*, which regulates the dynamic relationship between motion and contact forces rather than rigidly tracking a trajectory. In joint-space impedance control, each joint behaves as a virtual spring-damper system:

$$\boldsymbol{\tau} = K_q(\boldsymbol{q}_d - \boldsymbol{q}) + D_q(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}) \tag{2.2}$$

where K_q and D_q are diagonal stiffness and damping matrices, and $\boldsymbol{q}_d$, $\boldsymbol{q}$ are the desired and actual joint positions [11]. By tuning K_q and D_q, the engineer controls how compliantly each

joint responds to external forces: low stiffness yields soft, accommodating behavior, while high stiffness approximates rigid position control. Joint-space impedance is particularly important for safe deployment of dexterous hands, as it limits contact forces on a per-joint basis, preventing damage to the hand, the object, or the environment during grasping. Both the FR3 arm and the Agile Hand used in this work provide link-side torque sensing on every actuated joint, where the FR3 uses integrated joint torque sensors and the Agile Hand uses strain gauges mounted directly on the joint mechanisms. This enables joint-space impedance control on all 23 DoFs without external force-torque sensors. For real-world deployment, joint-space impedance provides a natural safety layer between the learned policy, which outputs joint position targets, and the physical actuators.

2.2 Classical Grasp Planning and Dexterous Manipulation

Before the advent of learning-based methods, dexterous manipulation relied on analytical grasp planning, predefined *manipulation primitives*, which are hand-coded motion sequences for specific subtasks such as "close fingers to power grasp," "rotate object using thumb opposition," or "slide object along palm," and carefully engineered control pipelines. This section reviews these classical approaches and their limitations, which motivate the shift to data-driven methods.

Analytical grasp planning. The classical theory of robotic grasping frames the problem as finding contact points on an object's surface that satisfy *force closure*, meaning the ability to resist arbitrary external wrenches through the contact forces alone [13, 14]. Given a known 3D object model and friction parameters, analytical planners search for contact configurations that optimize a quality metric such as the largest wrench ball inscribable in the Grasp Wrench Space (GWS) [14]. These methods work well for simple geometries and known objects, but require accurate object models, known friction coefficients, and precise contact modeling, which are assumptions that rarely hold in unstructured environments.

Traditional manipulation pipelines. Classical dexterous manipulation systems are typically organized as sequential pipelines: (1) perceive the object (e.g., via 3D scanning or template matching), (2) plan a grasp using analytical metrics or a precomputed grasp database, (3) execute the grasp using trajectory planning and impedance control, and (4) perform post-grasp manipulation via predefined manipulation primitives. Each stage is engineered independently, with hand-tuned parameters for the specific robot, gripper, and object set [11, 13]. For parallel-jaw grippers, this approach has been highly successful in structured industrial settings. However, extending it to multi-fingered hands is substantially harder: for a hand with n_h joints, the grasp planning problem involves searching over $\mathrm{SE}(3) \times \mathbb{R}^{n_h}$, making exhaustive search intractable. In practice, classical systems for dexterous hands rely on predefined grasp taxonomies (e.g., power grasp, precision grasp, tripod grasp) with manually specified finger configurations for each object category.

Limitations of classical approaches. The fundamental limitation of these methods is their dependence on accurate models: of the object geometry, of the contact physics, and of the robot kinematics and dynamics. When any of these models is inaccurate, as is inevitable with novel objects, deformable materials, or wear on the robot hardware, the planned grasps fail. Moreover, classical pipelines struggle with the *contact-rich* nature of dexterous manipulation: multi-finger grasping involves complex, discontinuous contact dynamics that are difficult to

model analytically and expensive to simulate online [13]. Each stage of the pipeline introduces errors that compound downstream, and the lack of closed-loop feedback between perception and action means the system cannot recover from unexpected perturbations.

The shift to data-driven methods. These limitations have driven a broad shift toward learning-based approaches, which can acquire manipulation skills directly from experience without explicit modeling of contact physics or object geometry [1]. Data-driven grasp planning was an early success: DexGraspNet [15] generated 1.32 million grasps for over 5,000 objects using a differentiable force-closure estimator, and Wu et al. [2] demonstrated 86.7% grasp success on novel objects using a learned generative model. More recently, end-to-end RL has emerged as a particularly promising paradigm for dexterous manipulation, as it can learn both the grasp strategy and the reactive control policy jointly, thereby sidestepping the brittle hand-offs of traditional pipelines. UniDexGrasp++ [16] exemplifies this trend, decomposing the problem into a learned grasp proposal stage followed by goal-conditioned RL execution, achieving generalization across novel objects that would be infeasible with analytical planning.

2.3 Reinforcement Learning

RL provides a framework for learning sequential decision-making through trial and error [17]. An RL problem is formalized as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s'|s,a)$ the transition dynamics, $R(s,a)$ the reward function, and $\gamma \in [0,1)$ the discount factor. The agent seeks a policy $\pi(a|s)$ that maximizes the expected cumulative discounted reward $J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right]$.

Policy gradient methods. For continuous action spaces typical of robotics, policy gradient methods parameterize the policy π_θ and optimize J by gradient ascent: $\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t\right]$, where $\hat{A}_t$ is an estimate of the advantage function. Trust Region Policy Optimization (TRPO) [18] constrains the policy update to a trust region to ensure monotonic improvement. PPO [19] simplifies this by using a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{2.3}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and ϵ is the clipping parameter. PPO has become the dominant algorithm for robotic RL due to its stability and ease of implementation. It is used throughout the dexterous manipulation literature [3, 20, 21] and in this work for training the privileged teacher policy.

Deep RL for robotics. The combination of deep neural networks with RL, known as DRL, has enabled learning of complex behaviors from high-dimensional sensory inputs [22]. In robotics, DRL has been applied to locomotion, navigation, and manipulation tasks [1]. However, robotics poses specific challenges: real-world interaction is slow, expensive, and potentially destructive; reward functions are difficult to specify; and the dynamics are often partially observable. These challenges have driven the adoption of simulation-based training with subsequent transfer to real hardware.

2.4 Sim-to-Real Transfer

Training RL policies entirely in simulation circumvents the cost and safety issues of real-world learning, but introduces the *sim-to-real gap*, referring to discrepancies between simulated and real physics, perception, and actuation.

Domain randomization. The most widely adopted strategy for bridging the sim-to-real gap is Domain Randomization (DR), which varies simulation parameters (friction, mass, lighting, sensor noise, actuator delays) across training episodes so that the policy learns to be robust to a wide distribution of conditions [23]. The hypothesis, illustrated in fig. 2.1, is that if the real world falls within the distribution of randomized simulations, the policy will generalize without fine-tuning. Peng et al. [24] demonstrated that randomizing dynamics parameters enables zero-shot sim-to-real transfer for locomotion and manipulation tasks.

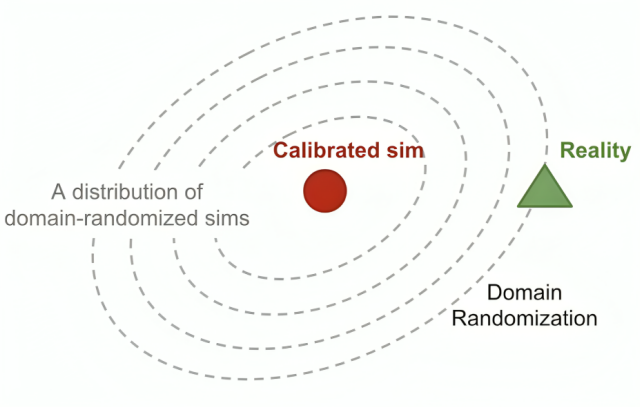

Figure 2.1: Illustration of domain randomization. A single calibrated simulation (red circle) may not match reality (green triangle), but a sufficiently wide distribution of randomized simulations (dashed ellipses) can encompass the real-world conditions, enabling zero-shot transfer. Figure adapted from [23].

Automatic domain randomization. Manual selection of randomization ranges is tedious and error-prone. Automatic Domain Randomization (ADR) automates this process by gradually widening parameter ranges based on policy performance [25]. The method maintains lower and upper bounds for each randomization dimension and adjusts them using performance thresholds, creating a natural curriculum from easy to difficult conditions. Handa et al. [3] extended ADR to vectorized GPU-based environments (VADR), enabling efficient scaling to thousands of parallel simulation instances.

System identification. An alternative to domain randomization is *system identification*, which fits simulation parameters to match real-world behavior. Lin et al. [4] introduced an automated real-to-sim tuning module that uses less than four minutes of real-world data to calibrate simulator parameters, significantly reducing the sim-to-real gap for bimanual dexterous manipulation on humanoid robots. Hybrid approaches that combine system identification with moderate domain randomization often achieve the best transfer results.

2.5 Imitation Learning and Policy Distillation

While RL can discover optimal behaviors from scratch, it requires extensive exploration and careful reward engineering. Imitation Learning (IL) offers an alternative by learning from expert demonstrations.

Policy distillation. The term *policy distillation* refers to the process of transferring the knowledge encoded in one policy (the *teacher*) into another policy (the *student*) via supervised learning. Originally introduced in the context of model compression [26], where a large, expressive network is distilled into a smaller, faster one, the concept has been widely adopted in robotics for a different purpose: bridging the gap between what is available during training and what is available during deployment. In simulation-based robot learning, the teacher policy can be trained with access to information that is easy to obtain in simulation, such as exact object poses, contact forces, and full kinematic state, but impossible or impractical to measure on a real robot. Policy distillation allows this privileged knowledge to be compressed into a student policy that relies only on deployable sensor modalities such as camera images. The distillation itself is an IL problem: the student learns to imitate the teacher's actions, using the teacher as an expert demonstrator. The key distinction from standard IL is that the expert is not a human but a trained policy, which can be queried at arbitrary states and at any scale, enabling interactive distillation methods such as DAgger that would be impractical with a human expert.

Behavioral cloning. The simplest form of IL is Behavioral Cloning (BC), which treats policy learning as supervised regression: given a dataset of state-action pairs $\{(s_i, a_i^*)\}$ collected under an expert policy π^*, the learner minimizes $\mathbb{E}[\|\pi_\theta(s) - a^*\|^2]$. Pomerleau [27] pioneered this approach with ALVINN, an autonomous driving system trained from human demonstrations. BC is straightforward but suffers from *compounding errors*: small deviations from the expert's state distribution lead to states not covered by the training data, causing the policy to drift further and fail [7].

DAgger. Ross et al. [7] introduced *Dataset Aggregation* (DAgger) to address the compounding error problem. DAgger iteratively executes the current student policy $\hat{\pi}$ in the environment, queries the expert π^* for the correct action at each visited state, and aggregates the new data with the existing dataset for retraining. After N iterations, DAgger provably finds a policy with performance bounded by $\epsilon_N + O(1/N)$, where ϵ_N is the best achievable loss on the aggregated dataset. DAgger is the distillation method used in DextrAH-RGB [5] for transferring privileged grasping policies to vision-based students.

SafeDAgger. While DAgger solves the distribution shift problem, it queries the expert at every visited state, which is wasteful when the student already performs well. More critically, it does not prevent the student from visiting dangerous states during rollouts. Zhang and Cho [9] proposed *SafeDAgger*, which trains a binary safety classifier $D_\psi(s) \in \{0, 1\}$ alongside the student policy. The classifier predicts whether the student's action at state s is "safe" (close to the expert's action) or "dangerous" (far from it). During data collection, the system switches from the student to the expert whenever D_ψ predicts danger, and back to the student when it predicts safety. This mechanism reduces unnecessary expert queries and prevents the student from executing actions in states where it is likely to fail, creating an implicit curriculum from safe to progressively more challenging states.

DexSafeDagger. Zhang [10] proposed DexSafeDagger, an extension of SafeDAgger for dexterous manipulation that introduces a *Q-value-based risk predictor*. This work has been submitted for publication but has not yet undergone peer review. Rather than relying solely on action disagreement between teacher and student, DexSafeDagger uses a learned Q-value function from the teacher's RL training to estimate the risk of each state. The teacher intervenes when either the action disagreement exceeds a threshold or the Q-value indicates a high-risk state. DexSafeDagger is particularly relevant for dexterous hands, where unsafe exploration can cause hardware damage or lead to irrecoverable failure states such as dropping the object.

Privileged teacher-student distillation. A powerful paradigm that combines RL and IL is *privileged teacher-student distillation* [28]. The idea is to train a teacher policy via RL with access to privileged information, including ground-truth object poses, contact forces, and full state vectors, that would be unavailable on a real robot. This teacher can learn highly effective behaviors because its observation space is compact, low-noise, and fully informative. The teacher's behavior is then distilled into a student policy that operates from realistic, deployable sensor inputs (e.g., RGB images). This decomposition separates the challenges of learning *what* to do (handled by RL with privileged info) from learning *how* to perceive (handled by distillation from raw sensory data). Lum et al. [6] and Singh et al. [5] demonstrated this approach for dexterous arm-hand grasping, with the teacher using geometric fabrics and the student operating from depth or RGB images. Huang et al. [29] further showed that the student can surpass the teacher's performance by combining imitation pre-training with online RL fine-tuning, addressing the constrained performance ceiling of pure imitation.

2.6 Dexterous Grasping and Manipulation

Dexterous manipulation with multi-fingered hands has advanced rapidly in recent years, driven by improvements in simulation, GPU-accelerated training, and sim-to-real transfer.

Early milestones. Kumar et al. [30] demonstrated model-based RL for dexterous manipulation using locally linear models, achieving contact-rich tasks on a 24-DoF hand. The landmark work by Andrychowicz et al. [20] at OpenAI showed that a Shadow Dexterous Hand could learn to solve a Rubik's Cube through massive-scale RL in simulation with domain randomization, establishing that sim-to-real transfer is viable even for extremely complex manipulation.

GPU-accelerated training. The introduction of Isaac Gym [31] enabled thousands of parallel simulation environments on a single GPU, reducing training times from weeks to hours. Handa et al. [3] leveraged this with DeXtreme, combining vectorized ADR with a vision-based pose estimator to achieve robust in-hand cube reorientation on a real Allegro Hand using only commodity hardware and no motion capture. Petrenko et al. [21] introduced DexPBT, which applies population-based training to dexterous tasks, automatically discovering reward weights and hyperparameters that yield diverse manipulation skills including bimanual coordination.

Arm-hand grasping. While much early work focused on in-hand manipulation with a fixed wrist, practical applications require coordinated arm-hand control, encompassing reaching for objects, grasping them, and transporting them. DextrAH-G [6] addressed this with a

23-motor arm-hand system, combining geometric fabrics [8] with RL to achieve fast, safe dexterous grasping from depth images. Geometric fabrics provide a structured control layer that guarantees collision avoidance and smooth motions, while the RL policy operates on top of this layer to learn task-specific behavior. DextrAH-RGB [5] extended this to RGB-only input, demonstrating the first successful sim-to-real transfer of an end-to-end RGB-based dexterous grasping policy. The key innovations include training the privileged teacher as a FGP and using photorealistic tiled rendering for distillation.

Positioning of this work. This thesis builds on the DextrAH-RGB framework but makes two significant departures. First, the privileged teacher is trained as a standard RL policy without geometric fabrics, simplifying the system and using Isaac Lab's direct environment API (`DirectRLEnv`) with custom assets. Second, the distillation step replaces DAgger with SafeDAgger, incorporating safety-aware intervention during training. This combination has not been explored in prior work on dexterous grasping.

2.7 Simulation Frameworks

Modern robot learning relies heavily on physics simulators for data generation.

Isaac Gym and Isaac Lab. Isaac Gym [31] introduced GPU-accelerated rigid-body and articulated-body simulation with end-to-end GPU data flow, eliminating the CPU-GPU bottleneck that limited earlier simulators. It supports thousands of parallel environments, enabling billion-step RL training in hours. Isaac Lab (formerly Orbit [32]) builds on Isaac Sim and provides a modular, extensible framework for robot learning with two environment design paradigms: a direct API for maximum control, and a *manager-based* API that structures environments into observation, action, reward, termination, and curriculum managers. This thesis uses the direct API (`DirectRLEnv`), which provides full transparency over the simulation loop and matches the paradigm used in the published DextrAH-RGB implementation.

Chapter 3

Methods

This chapter formalizes the problem of learning a vision-based dexterous grasping policy and presents the mathematical framework underlying the two-stage privileged teacher–student pipeline. section 3.1 states the problem as a partially observable decision process. section 3.2 derives the privileged teacher training objective. section 3.3 introduces the distillation framework, covering DAgger, SafeDAgger, and DexSafeDagger.

3.1 Problem Formulation

3.1.1 Dexterous Grasping as a POMDP

The dexterous grasping task is modeled as a POMDP defined by the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R, \gamma), \tag{3.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{O}$ the observation space, $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ the transition function, $O : \mathcal{S} \to \Delta(\mathcal{O})$ the observation function, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ the reward function, and $\gamma \in [0, 1)$ the discount factor.

The full state $s_t \in \mathcal{S}$ includes the joint configuration and velocities of the arm–hand system, the object pose, velocity, and physical properties, as well as contact information. At deployment, the agent does not have access to s_t directly; instead, it receives an observation $o_t \sim O(\cdot|s_t)$ consisting of proprioceptive measurements and camera images. The goal is to find a policy $\pi : \mathcal{O} \to \Delta(\mathcal{A})$ that maximizes the expected discounted return

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t) \right]. \tag{3.2}$$

3.1.2 Privileged vs. Partial Observability

Learning directly under partial observability is difficult: the policy must simultaneously learn *what* to do (the manipulation skill) and *how* to perceive (extracting task-relevant information from raw sensor data). The privileged teacher–student paradigm decouples these two challenges by introducing two distinct observation functions:

$$o_t^{\text{teacher}} = O^{\text{priv}}(s_t), \tag{3.3}$$

$$o_t^{\text{student}} = O^{\text{partial}}(s_t), \tag{3.4}$$

where O^{priv} is a deterministic, information-complete mapping that provides the teacher with the full state (joint states, object pose, contact forces), and O^{partial} is the realistic observation function that provides the student with only proprioceptive measurements and RGB images.

The two-stage pipeline is then:

1. **Stage 1:** Train a privileged teacher π^* that maps $o_t^{\text{teacher}} \mapsto a_t$ by solving the MDP induced by O^{priv}.

2. **Stage 2:** Distill π^* into a student $\hat{\pi}$ that maps $o_t^{\text{student}} \mapsto a_t$ via imitation learning.

3.2 Privileged Teacher Training

Since the teacher receives the full state, its training reduces to an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The teacher is trained via PPO with an asymmetric actor–critic architecture.

3.2.1 Asymmetric Actor–Critic

The actor (policy) and critic (value function) receive different observation vectors. Let o_t^{actor} denote the teacher's actor observation and o_t^{critic} the critic observation, where o_t^{critic} may contain additional privileged information (e.g., object velocities, contact forces) beyond o_t^{actor}. The actor parameterizes a Gaussian policy

$$\pi_\theta(a_t | o_t^{\text{actor}}) = \mathcal{N}\left(\mu_\theta(o_t^{\text{actor}}),\ \sigma_\theta^2\right), \tag{3.5}$$

while the critic estimates the state–value function

$$V_\phi(o_t^{\text{critic}}) \approx \mathbb{E}_\pi\left[\sum_{k=0}^{T-t} \gamma^k R(s_{t+k}, a_{t+k}) \,\middle|\, s_t\right]. \tag{3.6}$$

This asymmetry allows the critic to use richer information for more accurate value estimates during training, while the actor learns a policy that depends only on the subset of state available at deployment (before distillation to the student).

3.2.2 PPO Objective

The teacher is optimized using the clipped PPO surrogate objective [19]:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \tag{3.7}$$

where $r_t(\theta) = \pi_\theta(a_t | o_t^{\text{actor}}) / \pi_{\theta_{\text{old}}}(a_t | o_t^{\text{actor}})$ is the importance sampling ratio, $\hat{A}_t$ is the generalized advantage estimate (GAE) [19], and ϵ is the clipping parameter.

The advantage is computed using the critic via

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = R(s_t, a_t) + \gamma V_\phi(o_{t+1}^{\text{critic}}) - V_\phi(o_t^{\text{critic}}), \tag{3.8}$$

where $\lambda \in [0, 1]$ is the GAE parameter that trades off bias and variance.

3.2.3 Reward Structure

The reward function decomposes as a weighted sum of K terms:

$$R(s_t, a_t) = \sum_{i=1}^{K} w_i \cdot r_i(s_t, a_t), \tag{3.9}$$

where each r_i captures a different aspect of the grasping task and $w_i \in \mathbb{R}$ is its weight. The terms can be broadly categorized into:

- **Approach rewards** that encourage the hand to move toward the object,
- **Contact rewards** that incentivize stable multi-finger contact,
- **Lift rewards** that reward raising the object to a goal height,
- **Regularization penalties** that discourage jerky or excessive actions.

The specific reward terms and their weights are detailed in the experimental setup (section 4.2.3).

3.2.4 Adaptive Domain Randomization

To produce a robust teacher policy that generalizes across varying physical conditions, the training environment applies ADR [25]. Let $\xi \in \Xi$ denote the vector of randomizable simulation parameters (e.g., friction, mass, actuator gains). ADR maintains an interval $[\xi_i^{\text{lo}}, \xi_i^{\text{hi}}]$ for each parameter ξ_i and uniformly samples $\xi_i \sim \mathcal{U}(\xi_i^{\text{lo}}, \xi_i^{\text{hi}})$ at each episode reset.

The intervals are widened according to a curriculum: when the policy's rolling success rate exceeds a threshold τ_{ADR}, a shared increment counter n is advanced, and each interval is linearly interpolated from its initial range toward its maximum range:

$$\xi_i^{\text{lo/hi}}(n) = \xi_i^{\text{start, lo/hi}} + \frac{n}{N_{\max}} \left(\xi_i^{\text{max, lo/hi}} - \xi_i^{\text{start, lo/hi}} \right), \tag{3.10}$$

where $N_{\max}$ is the total number of ADR increments. This creates a natural curriculum: early in training the policy sees near-nominal physics, and as it improves, the randomization range widens to cover progressively more challenging conditions.

3.3 Policy Distillation

Given a trained teacher π^* operating on privileged observations, the goal of distillation is to train a student policy $\hat{\pi}_\theta : \mathcal{O}^{\text{partial}} \to \mathcal{A}$ that imitates the teacher using only partial observations. The student minimizes the imitation loss

$$\mathcal{L}(\theta) = \mathbb{E}_{o_t^{\text{student}} \sim d_{\hat{\pi}}} \left[\left\| \hat{\pi}_\theta(o_t^{\text{student}}) - \pi^*(o_t^{\text{teacher}}) \right\|_2^2 \right], \tag{3.11}$$

where $d_{\hat{\pi}}$ is the state distribution induced by the student's own policy. When both teacher and student output Gaussian distributions, an alternative is the KL divergence loss:

$$\mathcal{L}_{\text{KL}}(\theta) = \mathbb{E}_{o_t^{\text{student}} \sim d_{\hat{\pi}}} \left[D_{\text{KL}}\big(\pi^*(\cdot | o_t^{\text{teacher}}) \parallel \hat{\pi}_\theta(\cdot | o_t^{\text{student}}) \big) \right], \tag{3.12}$$

which matches the full action distribution rather than only the mean, and has been found to outperform L2 regression in prior work on visuomotor distillation [5]. The key challenge is that the training distribution $d_{\hat{\pi}}$ shifts as the student policy changes, causing compounding errors if naive behavioral cloning is used.

3.3.1 DAgger

DAgger [7] addresses the distribution shift by iteratively collecting data under the student's own policy while labeling it with the teacher's actions. The procedure is formalized in algorithm 1.

Algorithm 1 DAgger [7]

```
Initialize dataset D ← ∅, student policy π̂_{θ_1}
for i = 1, ..., N do
    Set roll-in policy π_i = β_i π* + (1 − β_i) π̂_{θ_i}
    Collect trajectories {(o_t^student, s_t)} by executing π_i
    Label: a_t^* = π*(o_t^teacher) for each visited state
    Aggregate: D ← D ∪ {(o_t^student, a_t^*)}
    Train π̂_{θ_{i+1}} on D by minimizing eq. (3.11)
end for
return π̂_{θ_{N+1}}
```

The mixing coefficient $\beta_i \in [0,1]$ controls the fraction of teacher vs. student rollouts. In the simplest schedule, $\beta_1 = 1$ (pure teacher) and $\beta_i = 0$ for $i > 1$ (pure student). After N iterations, DAgger guarantees a policy with loss bounded by $\epsilon_N + O(1/N)$, where ϵ_N is the best achievable loss on $\mathcal{D}$.

3.3.2 SafeDAgger

While DAgger resolves distribution shift, it does not prevent the student from visiting dangerous states during data collection, nor does it prioritize teacher queries for states where the student is most likely to fail. SafeDAgger [9] augments DAgger with a learned *safety policy* that determines when the teacher must intervene.

Safety Policy

A binary classifier $D_\psi : \mathcal{O}^{\text{partial}} \to [0,1]$ is trained to predict whether the student's action at a given observation is safe. The safety label for each data point is defined as

$$y_t = \begin{cases} 1 & \text{(safe)} \quad \text{if } \left\| \hat{\pi}_\theta(o_t^{\text{student}}) - \pi^*(o_t^{\text{teacher}}) \right\|_2 \le \delta, \\ 0 & \text{(unsafe)} \quad \text{otherwise,} \end{cases} \tag{3.13}$$

where $\delta > 0$ is a threshold on the action disagreement between student and teacher. The classifier is trained via binary cross-entropy on the aggregated dataset:

$$\mathcal{L}_D(\psi) = -\mathbb{E}\left[y_t \log D_\psi(o_t) + (1 - y_t) \log(1 - D_\psi(o_t)) \right]. \tag{3.14}$$

Intervention Mechanism

During data collection, the safety policy switches control between the student and teacher at each time step:

$$a_t = \begin{cases} \hat{\pi}_\theta(o_t^{\text{student}}) & \text{if } D_\psi(o_t^{\text{student}}) \ge 0.5, \\ \pi^*(o_t^{\text{teacher}}) & \text{otherwise.} \end{cases} \tag{3.15}$$

This creates an adaptive, state-dependent mixing policy: the teacher intervenes precisely when the student is predicted to deviate significantly, while the student acts autonomously in states where it is competent. The resulting intervention rate

$$\beta_{\text{safe}} = \frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \mathbb{1}\left[D_\psi(o_t^{\text{student}}) < 0.5\right] \tag{3.16}$$

naturally decays as the student improves, providing an implicit curriculum from teacher-guided exploration to student-autonomous execution. The complete SafeDAgger procedure is given in algorithm 2.

Algorithm 2 SafeDAgger [9]. Blue lines indicate modifications relative to DAgger (algorithm 1).

```
Initialize dataset D ← ∅, student π̂_{θ_1}, safety policy D_{ψ_1}
for i = 1, ..., N do
    for each time step t in rollout do
        if D_{ψ_i}(o_t^student) ≥ 0.5 then
            Execute a_t = π̂_{θ_i}(o_t^student)                ▷ Student acts
        else
            Execute a_t = π*(o_t^teacher)                      ▷ Teacher intervenes
        end if
        Query teacher: a*_t = π*(o_t^teacher)
        Compute safety label y_t via eq. (3.13)
        D ← D ∪ {(o_t^student, a*_t, y_t)}
    end for
    Update π̂_{θ_{i+1}} by minimizing eq. (3.11) on D
    Update D_{ψ_{i+1}} by minimizing eq. (3.14) on D
end for
return π̂_{θ_{N+1}}
```

DexSafeDagger: Q-Value Risk Prediction

Zhang [10] proposed an extension of SafeDAgger for dexterous manipulation that replaces the action-disagreement classifier with a *Q-value-based risk predictor*. This work, referred to as DexSafeDagger, has been submitted for publication but has not yet undergone peer review. Rather than relying solely on the L2 distance between student and teacher actions, DexSafeDagger uses the teacher's learned value function V_ϕ to estimate the risk of the current state.

The intervention criterion becomes

$$a_t = \begin{cases} \hat{\pi}_\theta(o_t^{\text{student}}) & \text{if } V_\phi(s_t) \geq V_{\text{safe}} \text{ and } \|e_t\|_2 \leq \delta, \\ \pi^*(o_t^{\text{teacher}}) & \text{otherwise,} \end{cases} \tag{3.17}$$

where $e_t = \hat{\pi}_\theta(o_t^{\text{student}}) - \pi^*(o_t^{\text{teacher}})$ is the action error, V_{safe} is a value threshold, and δ is the action disagreement threshold. The dual criterion captures two complementary failure modes: (1) states where the student's action deviates from the teacher (behavioral error), and (2) states that the teacher's own value function identifies as high-risk (environmental danger).

Chapter 4

Experiments and Results

This chapter describes the experimental realization of the methods introduced in chapter 3. It begins with the hardware and simulation setup (section 4.1), details the teacher and student policy implementations (sections 4.2 and 4.3), defines the evaluation protocol (section 4.4), and presents results for both teacher training (section 4.5) and distillation (section 4.6).

4.1 Experimental Setup

4.1.1 Hardware Platform

The robotic platform consists of a FR3 7-DoF arm equipped with an Agile Hand, forming a 23-DoF arm–hand system.

Franka Research 3. The FR3 is a torque-controlled collaborative robot arm with seven revolute joints and integrated joint torque sensors on every axis. The arm provides Cartesian impedance control capabilities, though in this work all joints are controlled via position targets with simulated PD controllers. The factory torque limits are 87 Nm for joints 1–4 and 12 Nm for joints 5–7.

Agile Hand. The Agile Hand, manufactured by Agile Robots SE, is a 16-DoF anthropomorphic robotic hand composed of five identical, modularly designed fingers with an aluminum open skeleton structure and injection-molded plastic shells. The hand weighs 1.5 kg and has 21 joints total, of which 16 are independently actuated (one coupled joint per finger). Each finger has four joints: a metacarpophalangeal (MCP) pitch joint (0° to 75°), an MCP yaw joint (±15°), a proximal interphalangeal (PIP) joint (0° to 85°), and a distal interphalangeal (DIP) joint (0° to 90°, coupled to PIP). The opposing thumb adds a rotation DoF with a range of 60°. The hand provides 10 N active fingertip force, 360°/s finger joint velocity, and EtherCAT-based real-time communication at 1 kHz. The thumb rotation joint operates at a significantly lower velocity (≈12 deg/s), which is a key constraint for sim-to-real transfer (see section 4.8). Joint torque and position sensors are integrated in every actuated joint, enabling active compliant control. The combined arm–hand system provides 23 actuated DoFs: 7 arm joints plus 16 hand joints.

Sensor setup.

4.1.2 Simulation Environment

All training and distillation is performed in NVIDIA Isaac Lab [32], a GPU-accelerated simulation framework built on Isaac Sim and PhysX 5.

Direct environment API. Isaac Lab provides two paradigms for environment design: a *manager-based* API and a *direct* API (`DirectRLEnv`). This work uses the direct API, which provides full transparency over the simulation loop: observations, rewards, resets, and domain randomization are implemented explicitly in Python rather than delegated to abstracted manager modules. This is the same paradigm used in the published DextrAH-RGB implementation [5] and enables fine-grained control over reward computation, ADR scheduling, and physics parameter manipulation at each time step.

Physics configuration. The simulation runs at 120 Hz with a control frequency of 60 Hz (decimation factor of 2). table 4.1 summarizes the key PhysX solver parameters.

Table 4.1: PhysX simulation parameters.

Parameter	Value
Simulation frequency	120 Hz
Control frequency	60 Hz
Solver position iterations	8
Solver velocity iterations	0
Max depenetration velocity	30.0 m/s
Bounce threshold velocity	0.2 m/s
Friction offset threshold	0.04 m
GPU max rigid patch count	10×2^{15}

Asset integration. The FR3 arm and Agile Hand are combined into a single articulated asset in Universal Scene Description (USD) format. Joint physics parameters (damping, stiffness, effort limits) are configured both in the USD file and via Isaac Lab's actuator API. table 4.2 lists the actuator parameters used for the final training configuration.

Table 4.2: Actuator parameters for the FR3 + Agile Hand system.

Joint Group	Stiffness	Damping	Effort Limit (Nm)
Arm joints 1–4	200	40	100
Arm joints 5–7	200	40	50
Finger MCP pitch	10.0	6.0	2.0
Finger MCP yaw	10.0	6.0	2.0
Finger PIP	10.0	6.0	2.0
Thumb rotation	20.0	2.0	2.0

Contact physics. The fingertip material uses static friction 2.0, dynamic friction 1.5 at nominal conditions, with the global friction combine mode set to `max` so that the higher rubber fingertip friction dominates at the contact interface. Object materials use static and dynamic friction of 1.0. These values are subject to ADR during training (see section 4.2.6).

4.1.3 Object Set

Training uses a set of 13 objects from the VisDex dataset, covering a range of geometries and sizes:

The objects include: mario, milk pot, turtle candle holder, toy bagger, teddy bear, closed fist, basketball shoe, chicken head in car, elephant toy, train, homer, toy cow, and plane. Object meshes are stored as individual USD assets. At episode reset, each parallel environment is assigned one object uniformly at random, and the object's mass, friction, and scale are randomized within the ADR ranges.

4.1.4 Episode Structure

Each episode has a maximum duration of 10 s (600 steps at 60 Hz) for teacher training and 5 s (300 steps) for student distillation. At the start of each episode, the object is placed on a table surface with randomized position and orientation, and the robot is initialized to a fixed home configuration. The arm is initialized above the object in a top-down approach pose to facilitate contact.

An episode terminates early if any of the following conditions are met:

- The object leaves the workspace bounds (`object_out_of_bound`).
- The hand moves too far from the object (`hand_too_far`).
- A harmful collision is detected (`harmful_collision`).
- The palm orientation flips beyond a threshold (`palm_flipped`).

Early terminations incur a penalty of -1.0 on the reward to discourage the policy from exploiting short episodes.

4.2 Teacher Policy Implementation

4.2.1 Observation Space

The teacher uses an asymmetric actor–critic architecture with different observations for actor and critic, as described in section 3.2.1. table 4.3 details the actor observation vector.

The critic observation additionally includes object velocities (6D), contact forces, contact torques, and the teacher one-hot flag, yielding a larger observation vector.

4.2.2 Action Space

The action space is 23-dimensional, consisting of joint position targets for all 7 arm joints and 16 hand joints. Actions are scaled from the policy's normalized output $a_t \in [-1, 1]^{23}$ to absolute joint position targets using per-joint scaling factors. No geometric fabrics or motion primitives are used; the policy directly commands joint positions, which distinguishes this work from the FGP-based approach in DextrAH-RGB [5].

Table 4.3: Teacher actor observation space.

Component	Dim	Description
Joint positions	23	Arm (7) + hand (16) joint angles
Joint velocities	23	Arm (7) + hand (16) joint velocities
Hand position	3	Palm center position in world frame
Hand velocity	3	Palm center linear velocity
Object position	3	Object center of mass position
Object orientation	4	Object orientation (quaternion)
Object goal position	3	Target lift height
Object scale	1	Randomized object scale factor
Previous action	23	Action applied at $t-1$
Teacher one-hot	1	Fixed $= 1$ (compatibility placeholder)
Total	**87**	

4.2.3 Reward Design

The reward function follows the structure defined in eq. (3.9). table 4.4 lists all reward terms, their formulations, and weights used in the final training configuration.

Table 4.4: Reward terms and weights for teacher training.

Term	Formulation	Weight
Hand-to-object distance	$\exp(-\kappa \cdot \lVert p_{\text{hand}} - p_{\text{obj}} \rVert)$	4.0
Palm direction alignment	$\max(0, \hat{n}_{\text{palm}} \cdot \hat{d}_{\text{obj}})$	0.5
Hand-object contact	$\min(n_{\text{contacts}}, n_{\max})/n_{\max}$	8.0
Good grasp	$\mathbb{1}[\text{contact mask}] \cdot \text{grasp quality}$	6.0
Object-to-goal (lift)	$w_{\text{lift}} \cdot \exp(-\kappa_l \cdot \lvert z_{\text{obj}} - z_{\text{goal}} \rvert) \cdot \mathbb{1}[\text{contact}]$	1.0
Finger curl regularization	$-\lVert \boldsymbol{q}_{\text{fingers}} - \boldsymbol{q}_{\text{curl}} \rVert^2$	0.5
Action rate penalty	$-\lVert a_t - a_{t-1} \rVert^2$ (fingers $\times 2.5$)	0.1

The reward design underwent extensive iteration. Key design decisions include:

- **Contact-gated lift reward:** The lift term is multiplied by a contact mask, ensuring the policy only receives lift reward when actually grasping the object, not when the object is accidentally launched upward.
- **Finger action rate penalty scaling:** Hand joints are penalized 2.5× more than arm joints for jerky actions, encouraging smooth finger motions critical for stable grasps.
- **Finger curl regularization:** A weak penalty encourages fingers toward a pre-grasp curl pose, providing an inductive bias for power grasps without constraining the policy.

4.2.4 Network Architecture

The teacher policy uses an Long Short-Term Memory (LSTM)-based recurrent network implemented in rl_games. The actor consists of an MLP encoder followed by an LSTM layer with 1024 hidden units, followed by an output head mapping to the 23-dimensional action space. The critic has an identical architecture but receives the larger critic observation.

4.2.5 Training Configuration

table 4.5 summarizes the PPO hyperparameters used for teacher training.

Table 4.5: PPO hyperparameters for teacher training.

Hyperparameter	Value
Learning rate	1×10^{-4}
Clip parameter ϵ	0.2
Mini-epochs per update	4
Horizon length	16 steps
Minibatch size	2048
Discount factor γ	0.99
GAE λ	0.95
Number of environments	512
Max training epochs	20,000

Training is performed headless (no rendering) on a single NVIDIA GPU.

4.2.6 Adaptive Domain Randomization

The ADR curriculum follows the formulation in section 3.2.4 with $N_{\max} = 50$ increments and a success threshold $\tau_{\text{ADR}} = 0.4$. table 4.6 lists the key randomized parameters and their start/max ranges.

Table 4.6: Adaptive Domain Randomization parameters.

Parameter	Start Range	Max Range
Object mass scale	$[1.0, 1.0]$	$[0.5, 2.0]$
Object friction (static)	$[1.0, 1.0]$	$[0.5, 1.5]$
Robot restitution	$[1.0, 1.0]$	$[0.8, 1.0]$
Arm stiffness scale	$[1.0, 1.0]$	$[0.5, 2.0]$
Arm damping scale	$[1.0, 1.0]$	$[0.5, 2.0]$
Finger MCP pitch stiffness	$[1.0, 1.0]$	$[0.17, 1.0]$
Finger MCP pitch damping	$[1.0, 1.0]$	$[0.083, 1.0]$
Finger MCP yaw stiffness	$[1.0, 1.0]$	$[0.025, 1.0]$
Finger PIP stiffness	$[1.0, 1.0]$	$[0.02, 1.0]$
Thumb rotation velocity limit	$[20.0, 20.0]$	$[0.21, 20.0]$ rad/s

The ADR is designed with sim-to-real transfer in mind: the maximum ranges for finger joint stiffness and damping correspond to values measured on the real Agile Hand hardware using an Isaac Lab gain tuner. The thumb rotation velocity limit is ramped down toward the real hardware limit of approximately 12 deg/s (≈ 0.21 rad/s).

All ADR parameters share a single increment counter, meaning all parameters progress together. This simplifies the curriculum but does not allow per-parameter scheduling.

4.3 Student Policy Implementation

4.3.1 Student Observation Space

The student replaces the teacher's privileged object state with RGB image observations. The student observation consists of:

- **Proprioceptive state:** Joint positions (23D), joint velocities (23D), hand position (3D), hand velocity (3D), object goal position (3D), and previous action (23D).
- **Visual input:** Simulated stereo RGB images at 320×240 resolution from wrist-mounted cameras.

Crucially, the student does *not* receive the object position, orientation, scale, or contact information; these must instead be inferred from the visual input.

4.3.2 Student Network Architecture

The student uses a vision–proprioception fusion architecture:

1. **Visual encoder:** A ResNet-18 backbone processes each stereo image independently, producing a 32-dimensional feature vector per image.
2. **Cross-attention fusion:** A cross-attention transformer module fuses the two stereo features into a single 64-dimensional visual embedding.
3. **Policy head:** The visual embedding is concatenated with the proprioceptive state vector and fed into an LSTM with 512 hidden units, followed by a linear output layer producing the 23-dimensional action.

4.3.3 SafeDagger Distillation Configuration

The distillation follows the SafeDAgger procedure described in algorithm 2. The safety policy uses the per-environment L2 action error as the intervention criterion: for each environment, the teacher takes over when $\|e_t\|_2 > \delta$, with $\delta = 0.5$. The intervention rate β_{safe} (eq. (3.16)) is computed as the fraction of environments where the threshold is exceeded.

Key distillation hyperparameters:

- **Optimizer:** Adam, learning rate 1×10^{-4}
- **Distillation iterations:** 350,000 (default)
- **Number of environments:** 16 (with cameras enabled)
- **ADR:** Disabled during distillation (`enable_adr=False`)
- **Arm randomization:** Disabled (`disable_arm_randomization=True`)
- **Stereo rendering:** Enabled (`simulate_stereo=True`)

4.4 Evaluation Protocol

4.4.1 Metrics

The following metrics are used to evaluate both teacher and student policies:

GSR: Fraction of episodes where the object is lifted above the table surface and held with stable contact for at least 0.5 s (30 steps at 60 Hz). The hold requirement prevents counting accidental upward launches as successful grasps.

Unsafe Episode Rate (UER): Fraction of episodes containing at least one early termination due to an unsafe event. Unsafe events are classified into five categories: `object_out_of_bound`, `hand_too_far`, `harmful_collision`, `palm_flipped`, and `physics_instability` (simulation-only artifacts such as NaN joint states or velocity explosions, not relevant for real-world deployment).

Teacher Query Rate (TQR): For SafeDAgger, the fraction of time steps where the teacher provides the action (i.e., $D_\psi(o_t) < 0.5$). This quantifies the student's autonomy during distillation.

Imitation loss: The L2 action error between student and teacher, averaged over the evaluation batch.

4.4.2 Evaluation Procedure

Teacher policies are evaluated using `eval_teacher.py`, which supports both single-checkpoint and multi-teacher (per-object directory) evaluation. Student policies are evaluated using `eval_student.py`, which additionally reports per-object metrics, failure reason breakdown, and the hold-gated lift success criterion.

4.5 Teacher Training Results

4.5.1 Training Progression

The teacher policy was trained on 13 objects from the VisDex dataset using 512 parallel environments. Training proceeded in two phases:

1. **Initial training (16 envs, runs 1a–1o):** A series of iterative experiments to resolve physics stability issues, reward shaping, and early termination handling. With only 16 environments across 13 objects (≈1.2 envs per object), training exhibited high gradient variance and cyclic performance collapses.

2. **Scaled training (512 envs, run 1p):** Resumed from the best checkpoint of the initial phase with 512 environments (≈39 envs per object), yielding stable gradient estimates and consistent learning progress.

The policy reached ADR level 5 by epoch 11,124, demonstrating reliable grasping and lifting across all 13 objects. A subsequent extended run with 1024 environments reached ADR level 19 after 20,000 epochs, producing a stronger teacher for distillation.

4.5.2 Physics Stability Challenges

A central challenge in training was achieving stable contact physics for the Agile Hand. The key failure modes and their resolutions are summarized in table 4.7.

Table 4.7: Physics stability challenges encountered during teacher training.

Failure Mode	Root Cause	Resolution
Velocity explosion on every reset	Joint init position outside joint limits (thumb_rot)	Corrected init to $-20°$ (joint minimum)
Finger joints explode on contact	Effort limit 10 Nm (10× too high for hand joints)	Reduced to 2.0 Nm
Policy avoids contact	Contact causes simulation instability → early termination → negative reward	Physics stability overhaul: damping 3 → 6, effort limits reduced
Gradient oscillation	16 envs / 13 objects → ∼1 env per object	Scaled to 512 environments
Reward hacking (hover)	Dense rewards achievable without contact	Contact-gated lift reward, early termination penalty

4.5.3 Teacher Performance

table 4.8 summarizes the performance of the two teacher checkpoints used for distillation experiments.

Table 4.8: Teacher policy performance (multi-object, 13 VisDex objects). The ADR-19 teacher was evaluated over 80 episodes (16 envs × 5 rollouts) at ADR-0 physics.

Metric	ADR-5 Teacher	ADR-19 Teacher
ADR level reached	5	19
Training epochs	11,124	20,000
Number of environments	512	1,024
Mean episode reward	12,104	5,317
GSR (%)	–	96.25
UER (%)	–	53.75

Note that the ADR-19 teacher has a lower mean reward than the ADR-5 teacher because it operates under significantly more challenging physics conditions (wider randomization ranges). The lower reward reflects harder conditions, not worse policy quality. The teacher's 53.75% UER is dominated by `physics_instability` (55.8% of all failures), which are simulation-only artifacts (NaN joint states, velocity explosions) that cannot occur on real hardware. Excluding these, the real-world-relevant unsafe rate is approximately 24%, with the remaining failures split between `object_out_of_bound` (32.6%) and `harmful_collision` (11.6%).

4.6 Distillation Results

Three SafeDAgger distillation runs were performed, progressively increasing teacher strength and training duration. All use the 13-object VisDex set and identical student architectures.

4.6.1 SafeDAgger Distillation

table 4.9 compares the three distillation runs. Runs 1a and 1b used 100k iterations with 8 environments; run 1c extended to 350k iterations with 16 environments.

Table 4.9: SafeDAgger distillation results (during-training metrics, including teacher-assisted actions).

Metric	Run 1a (ADR-5, 100k)	Run 1b (ADR-19, 100k)	Run 1c (ADR-19, 350k)
Imitation loss (L2)	2.10	1.48	1.40
Lifted (%)	62.5	87.5	62.5
In goal (%)	37.5	62.5	37.5
β_{safe}	1.0	0.875	0.75
Num. environments	8	8	16

Effect of teacher strength. Comparing runs 1a and 1b (same iterations, different teachers), the stronger ADR-19 teacher yields a 40% relative improvement in lift rate (62.5% → 87.5%), a 66% improvement in goal rate (37.5% → 62.5%), and a 30% reduction in imitation loss (2.10 → 1.48).

Effect of training duration. Extending from 100k to 350k iterations (run 1c) further reduced the imitation loss (1.48 → 1.40) and the intervention rate (0.875 → 0.75), indicating continued learning. However, the during-training metrics are misleading because they include teacher-assisted actions, as the teacher still performs the grasping in 75% of environments.

Standalone student evaluation. To measure true student performance without teacher assistance, the run 1c checkpoint was evaluated using `eval_student.py` over 80 episodes (16 envs × 5 rollouts). table 4.10 compares the standalone student against the teacher ceiling.

Table 4.10: Standalone student vs. teacher performance (80 episodes, ADR-0 physics).

Metric	Teacher (ADR-19)	Student (run 1c)
GSR (%)	96.25	36.25
UER (%)	53.75	67.50
Failure breakdown (% of unsafe episodes):		
Physics instability	55.8	–
Harmful collision	11.6	40.7
Object out of bound	32.6	33.3
Palm flipped	0.0	1.9
Hand too far	0.0	0.0

The student achieves 36.25% standalone lift success, which represents a significant gap from the 96.25% teacher ceiling. The primary failure mode is `harmful_collision` (40.7% of failures vs. 11.6% for the teacher), indicating the student crashes into the table surface 3.5× more frequently. Object knockoff rates are comparable (∼33%), suggesting this is an inherent task difficulty rather than a student-specific weakness. Arm control is robust for both teacher and student (no `hand_too_far` failures).

4.6.2 DAgger Baseline Comparison

A central goal of this work is a controlled comparison between vanilla DAgger and SafeDAgger for distilling a privileged dexterous grasping teacher into a vision-based student policy. The original SafeDAgger paper [9] evaluated only in simulated autonomous driving (TORCS) and compared solely against vanilla DAgger, without systematic ablation of the safety threshold or application to contact-rich manipulation. The DexSafeDagger extension [10] adds Q-value-based risk gating, but no published comparison exists on the specific platform and object set used here. This section addresses both gaps by providing a head-to-head evaluation under matched conditions.

Experimental design. Both methods use the same ADR-19 teacher, 13-object VisDex set, identical student architecture (ResNet-18 + cross-attention + LSTM-512), 16 parallel environments, and 350k distillation iterations. The only difference is the intervention mechanism:

- **DAgger:** The student always executes its own actions. The teacher provides target action labels at every time step, and the student is trained with KL divergence loss (following the DextrAH-RGB finding that KL consistently outperforms L2 for action distillation [5]).
- **SafeDAgger:** The teacher takes over execution in environments where $\|e_t\|_2 > \delta$ (with $\delta = 0.5$), shielding the student from high-error states. The student is trained with L2 loss on all observed state–action pairs.

This design isolates the effect of SafeDAgger's selective intervention: does shielding the student from dangerous states improve safety at the cost of task performance, or does it also accelerate learning by providing a form of automated curriculum [9]?

Table 4.11: Head-to-head comparison of DAgger and SafeDAgger distillation (ADR-19 teacher, 13 objects, 350k iterations, 80 standalone evaluation episodes each).

Metric	DAgger (KL loss)	SafeDAgger (L2 loss)
Final GSR (%)	–	36.25
UER (%)	–	67.50
TQR (%)	100 (by design)	75.0
Imitation loss	–	1.40 (L2)
Harmful collisions (% of failures)	–	40.7
Object out of bound (% of failures)	–	33.3
Context switches (per episode)	0	–

The comparison addresses three specific hypotheses:

1. **Safety hypothesis:** SafeDAgger produces a student with fewer unsafe episodes (lower UER) during both training and standalone evaluation, particularly fewer harmful collisions.
2. **Performance hypothesis:** SafeDAgger's intervention mechanism may reduce standalone GSR by preventing the student from practicing in challenging states where it needs the most experience.
3. **Efficiency hypothesis:** SafeDAgger requires fewer total teacher queries (lower TQR) to reach comparable performance, as it selectively queries only on difficult states, creating an "automated curriculum" effect observed in the original SafeDAgger paper [9].

4.6.3 Safety Analysis

The standalone student evaluation reveals that the dominant failure mode is `harmful_collision` (40.7% of unsafe episodes), where the hand or arm contacts the table surface. This is 3.5× higher than the teacher (11.6%), indicating that the student has not fully learned the teacher's collision-avoidance behavior from visual input alone. Object knockoff (`object_out_of_bound`, 33.3%) is comparable between teacher and student, suggesting this is inherent to the grasp difficulty rather than a perception failure. Notably, no `hand_too_far` failures occur for either teacher or student, indicating that the arm positioning component of the policy transfers well through distillation.

4.6.4 Intervention Analysis

The SafeDAgger intervention rate β_{safe} provides insight into the student's learning progress. Over the course of training, β_{safe} decreased from 1.0 (100% teacher control) at 100k iterations to 0.75 at 350k iterations, indicating that the student progressively takes over more environments. However, at $\beta_{\text{safe}} = 0.75$ the teacher still controls 75% of environments, contributing to the gap between during-training metrics (62.5% lifted) and standalone evaluation (36.25% GSR). This gap highlights a key limitation of SafeDAgger's intervention mechanism: by shielding the student from difficult states, the teacher prevents the student from practicing precisely the situations where it needs the most experience.

4.7 Ablation Studies

4.7.1 Effect of Number of Training Environments

A critical finding from the teacher training experiments is the importance of sufficient parallel environments for multi-object training. With 16 environments across 13 objects, the policy exhibited cyclic performance collapses due to high gradient variance (∼1.2 envs per object per batch). Scaling to 512 environments (∼39 envs per object) produced stable, monotonic learning progress and enabled the first successful ADR progression. This suggests a minimum of ∼30 environments per object class is necessary for stable multi-object policy training.

4.7.2 Effect of Early Termination Penalty

Without an early termination penalty, the teacher policy learned to exploit short episodes: minimizing total negative reward by terminating quickly rather than attempting to grasp. Introducing a flat penalty of −1.0 per early termination eliminated this failure mode and was necessary for the policy to learn approach and contact behaviors. A higher penalty of −3.0 was initially tested but was reduced because it caused the policy to avoid any behavior that risked termination, including the necessary approach phase.

4.7.3 Effect of Teacher Strength on Distillation

The comparison between ADR-5 and ADR-19 teachers (table 4.9) demonstrates that teacher policy quality is a primary bottleneck for distillation performance. The stronger teacher

produces more consistent demonstrations, leading to lower imitation loss and higher task success. This aligns with the general finding in the imitation learning literature that demonstration quality matters more than demonstration quantity.

4.7.4 LSTM Checkpoint Resume for ADR Curriculum

An important negative finding from the sim-to-real curriculum experiments (exp_03) is that **resuming LSTM-based teacher policies from checkpoints does not work reliably for continued ADR training**. Three independent runs (1a–1c) resumed from the ADR-19 checkpoint and all degraded at ADR levels 16–18, with lift success dropping from ∼70% to ∼15–40%.

Investigation of the rl_games checkpoint format confirmed the root cause: while model weights, optimizer state, and running statistics are saved, **LSTM hidden states are reset to zero on load**. The LSTM must rebuild its temporal representations from scratch. At low ADR levels this is feasible (the task is easy enough), but at high ADR levels the difficulty margin is too thin for a cold-started LSTM to re-learn temporal context while also handling physics randomization. The original training succeeded because the LSTM built context *continuously* through ADR levels 0→19.

This finding motivated training the sim-to-real teacher from scratch (run 1d) rather than fine-tuning the existing checkpoint, incorporating all sim-to-real curriculum terms from the beginning.

4.7.5 Safety Threshold Sensitivity

A notable gap in the original SafeDAgger paper [9] is the lack of a systematic sensitivity analysis for the safety threshold δ. Zhang and Cho selected a single threshold ($\tau = 0.0025$ in their steering-angle domain) based on the error distribution of the initial supervised policy and did not vary it. This work addresses that gap by sweeping δ across five values: $\{0.1, 0.3, 0.5, 0.7, 1.0\}$.

For each threshold, a SafeDAgger distillation run is performed with the ADR-19 teacher, 16 environments, and 350k iterations. Lower δ values trigger teacher intervention more aggressively (more protective, fewer student-executed actions), while higher values allow the student to act autonomously in more states (less protective, more exploration). The extreme case $\delta \to \infty$ recovers vanilla DAgger (no intervention).

Table 4.12: Effect of safety threshold δ on SafeDAgger distillation performance (ADR-19 teacher, 350k iterations, standalone evaluation over 80 episodes).

δ	GSR (%)	UER (%)	TQR (%)	Harmful collisions (%)
0.1	–	–	–	–
0.3	–	–	–	–
0.5	36.25	67.50	75.0	40.7
0.7	–	–	–	–
1.0	–	–	–	–
∞ (DAgger)	–	–	100	–

This ablation reveals the safety–performance Pareto frontier for the dexterous grasping domain. If low-δ runs show lower UER but also lower GSR, the threshold acts as an explicit knob for trading safety against task performance, which is a critical design parameter for

eventual real-world deployment where hardware damage is costly. Conversely, if an intermediate δ simultaneously improves both metrics, it would confirm the "automated curriculum" hypothesis from Zhang and Cho: that focusing training on difficult states accelerates learning without sacrificing safety.

4.7.6 Per-Object Performance Breakdown

The 13 VisDex training objects span a wide range of geometries, from compact, graspable shapes such as the basketball shoe and teddy bear, to thin or irregular objects such as the plane and toy bagger. To identify which object categories benefit most from SafeDAgger's intervention mechanism, both the teacher and student policies are evaluated per-object over 80 episodes each (16 envs × 5 rollouts, with each environment assigned a single object).

Table 4.13: Per-object GSR (%) for teacher, SafeDAgger student, and DAgger student (80 evaluation episodes per object, ADR-0 physics).

Object	Teacher	SafeDAgger	DAgger
Mario	–	–	–
Milk pot	–	–	–
Turtle candle	–	–	–
Toy bagger	–	–	–
Teddy bear	–	–	–
Closed fist	–	–	–
Basketball shoe	–	–	–
Chicken in car	–	–	–
Elephant toy	–	–	–
Train	–	–	–
Homer	–	–	–
Toy cow	–	–	–
Plane	–	–	–
Mean	96.25	36.25	–

The per-object breakdown serves two purposes. First, it identifies which object geometries pose the greatest challenge for the vision-based student, informing whether the teacher–student gap is uniform or concentrated on specific shapes. Second, it reveals whether SafeDAgger's intervention is more beneficial for certain object categories, for example thin or irregular objects where collisions are more likely and teacher shielding may be more valuable.

4.7.7 Query Efficiency Analysis

A central claim of SafeDAgger is *query efficiency*: by selectively querying the teacher only on states where the student struggles, SafeDAgger should achieve comparable performance with fewer total teacher queries than vanilla DAgger [9]. In the original driving domain, SafeDAgger collected 70k targeted examples compared to DAgger's 390k uniformly sampled examples per iteration.

In the dexterous grasping setting, query efficiency is measured by the cumulative number of teacher-executed time steps over the course of distillation. For DAgger, this equals the total number of time steps (the teacher always provides labels, though the student always acts).

For SafeDAgger, this equals TQR × total steps, since the teacher only acts in environments where $\|e_t\|_2 > \delta$.

4.7.8 Learning Curve Comparison

To compare convergence dynamics, GSR and UER are evaluated at regular checkpoints throughout distillation for both DAgger and SafeDAgger. Checkpoints are saved every 50k iterations, yielding seven evaluation points per method (at 0k, 50k, 100k, 150k, 200k, 250k, 300k, and 350k iterations). Each checkpoint is evaluated standalone (without teacher intervention) over 80 episodes.

The learning curves address whether SafeDAgger's curriculum effect, which focuses training on difficult states, leads to faster initial convergence or whether DAgger's unrestricted exploration enables the student to eventually surpass SafeDAgger given sufficient iterations. The UER curves additionally reveal whether SafeDAgger students exhibit fewer unsafe behaviors throughout training, not just at the final checkpoint.

4.7.9 Effect of Distillation Duration

The comparison between runs 1b (100k iterations) and 1c (350k iterations) in table 4.9 suggests continued learning beyond 100k iterations. To determine whether the student has converged or would benefit from extended training, the learning curve analysis above is complemented by additional long-horizon runs at 500k and 750k iterations. If the GSR curve plateaus, the student has reached its capacity given the current architecture; if it continues to rise, the 350k baseline underestimates the method's potential.

4.8 Sim-to-Real Curriculum

To close the sim-to-real gap, a second-generation teacher is being trained from scratch with hardware-realistic constraints incorporated into the ADR curriculum.

4.8.1 Curriculum Design

The sim-to-real curriculum adds three categories of hardware constraints that are progressively introduced via ADR:

1. **Finger actuator gains:** Per-joint-group stiffness and damping scales, starting at nominal simulation values and widening toward hardware measurements. Initial attempts with hardware-exact gain targets (e.g., MCP pitch stiffness scale down to 0.17) caused policy collapse; relaxed ranges of $[0.1, 1.0]$ proved stable.

2. **Effort and velocity limits:** The thumb rotation velocity limit is ramped from 20 rad/s (simulation) toward 0.21 rad/s (≈12 deg/s, hardware limit). Arm effort limits are reduced toward FR3 factory specs (87 Nm for joints 1–4, 12 Nm for joints 5–7).

3. **Reward rebalancing:** The object-to-goal weight was doubled ($20 \rightarrow 40$) and a success bonus ($w = 10$) was added, since the original reward balance caused the policy to optimize for holding objects rather than delivering them to the goal height.

4.8.2 Preliminary Results

Training from scratch with the full sim-to-real curriculum (run 1d, 1024 envs) reached ADR level 12 of 50 after 45,000 epochs. The policy achieves ∼54% lift success but only ∼30% goal delivery, limited by the reward imbalance identified above. A follow-up run (1e) with rebalanced rewards and multi-GPU training (2048 envs across 2 GPUs) is in progress.

4.9 Summary of Findings

The key experimental findings are:

1. **Physics stability is the primary challenge** for training dexterous grasping policies with custom hand assets in Isaac Lab. Actuator parameters (effort limits, damping) must be carefully tuned to prevent simulation instabilities at contact, which otherwise cause the policy to learn contact-avoidance behaviors.
2. **Sufficient parallel environments are critical** for multi-object teacher training. A minimum of ∼30 environments per object class is needed to avoid gradient oscillation and cyclic collapses.
3. **Teacher policy quality is the primary bottleneck** for distillation. The ADR-19 teacher achieves 96.25% GSR, providing a high performance ceiling. Stronger teachers produce better students (40% relative improvement from ADR-5 to ADR-19).
4. **SafeDAgger produces functional but limited students:** 36.25% standalone GSR after 350k iterations, compared to the 96.25% teacher ceiling. The primary gap is in collision avoidance (3.5× more harmful collisions than the teacher).
5. **SafeDAgger's intervention mechanism is a double-edged sword:** It prevents unsafe student actions during training but also limits the student's exposure to challenging states, potentially slowing learning. The DAgger baseline comparison (section 4.6.2) and safety threshold sweep (section 4.7.5) are designed to quantify this trade-off and reveal the safety–performance Pareto frontier.
6. **LSTM policies cannot be fine-tuned from checkpoints** for continued ADR curriculum training, due to hidden state reset. Sim-to-real teachers must be trained from scratch with all curriculum terms included from the start.

The ablation studies additionally investigate:

- The safety threshold δ as an explicit knob for trading safety against performance (section 4.7.5), addressing a gap in the original SafeDAgger work [9].
- Per-object performance variation (section 4.7.6), revealing which geometries benefit most from teacher intervention during distillation.
- Query efficiency (section 4.7.7), testing whether SafeDAgger's selective querying achieves comparable GSR with fewer teacher interactions.
- Learning dynamics (section 4.7.8), comparing convergence speed and safety profiles throughout training, not just at the final checkpoint.

Chapter 5

Discussion and Conclusion

Appendix A

Appendix 1

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Bibliography

[1] Tang, C., Abbatematteo, B., Hu, J., Chandra, R., Martín-Martín, R., and Stone, P. "Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes". In: (Sept. 2024).

[2] Wu, A., Guo, M., and Liu, C. K. "Learning Diverse and Physically Feasible Dexterous Grasps with Generative Model and Bilevel Optimization". In: (Dec. 2022).

[3] Handa, A., Allshire, A., Makoviychuk, V., Petrenko, A., Singh, R., Liu, J., Makoviichuk, D., Wyk, K. V., Zhurkevich, A., Sundaralingam, B., Narang, Y., Lafleche, J.-F., Fox, D., and State, G. "DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality". In: (Jan. 2024). URL: https://arxiv.org/pdf/2210.13702.

[4] Lin, T., Sachdev, K., Fan, L., Malik, J., and Zhu, Y. "Sim-to-Real Reinforcement Learning for Vision-Based Dexterous Manipulation on Humanoids". In: (Sept. 2025).

[5] Singh, R., Allshire, A., Handa, A., Ratliff, N., and Wyk, K. V. "DextrAH-RGB: Visuomotor Policies to Grasp Anything with Dexterous Hands". In: (Feb. 2025).

[6] Lum, T. G. W., Matak, M., Makoviychuk, V., Handa, A., Allshire, A., Hermans, T., Ratliff, N. D., and Wyk, K. V. "DextrAH-G: Pixels-to-Action Dexterous Arm-Hand Grasping with Geometric Fabrics". In: (Oct. 2024).

[7] Ross, S., Gordon, G. J., and Bagnell, J. A. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". In: (Mar. 2011).

[8] Ratliff, N. and Wyk, K. V. "Fabrics: A Foundationally Stable Medium for Encoding Prior Experience". In: (Sept. 2023).

[9] Zhang, J. and Cho, K. "Query-Efficient Imitation Learning for End-to-End Autonomous Driving". In: (May 2016).

[10] Zhang, C. "DexSafeDagger: Safer Policy Distillation from RGB Observations Using Q-Value Failure Prediction–Guided Teacher Intervention". In: (Mar. 2026).

[11] Siciliano, B., Sciavicco, L., Villani, L., and Oriolo, G. *Robotics: Modelling, Planning and Control*. Springer, 2009. ISBN: 978-1-84628-641-4.

[12] Hogan, N. "Impedance Control: An Approach to Manipulation: Part I—Theory". In: *Journal of Dynamic Systems, Measurement, and Control* 107.1 (1985), pp. 1–7. DOI: 10.1115/1.3140702.

[13] Bicchi, A. and Kumar, V. "Robotic grasping and contact: a review". In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*. IEEE, 2000, pp. 348–353. ISBN: 0-7803-5886-4. DOI: 10.1109/ROBOT.2000.844081.

[14] Ferrari, C. and Canny, J. F. "Planning optimal grasps". In: *Proceedings 1992 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 1992, pp. 2290–2295. DOI: 10.1109/ROBOT.1992.219918.

[15] Wang, R., Zhang, J., Chen, J., Xu, Y., Li, P., Liu, T., and Wang, H. "DexGraspNet: A Large-Scale Robotic Dexterous Grasp Dataset for General Objects Based on Simulation". In: (Mar. 2023).

[16] Wan, W., Geng, H., Liu, Y., Shan, Z., Yang, Y., Yi, L., and Wang, H. "UniDexGrasp++: Improving Dexterous Grasping Policy Learning via Geometry-aware Curriculum and Iterative Generalist-Specialist Learning". In: (Apr. 2023).

[17] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018. ISBN: 978-0-262-03924-6.

[18] Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. "Trust Region Policy Optimization". In: (Feb. 2015).

[19] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. "Proximal Policy Optimization Algorithms". In: (July 2017).

[20] Andrychowicz, M., Baker, B., Chociej, M., Jozefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A., Schneider, J., Sidor, S., Tobin, J., Welinder, P., Weng, L., and Zaremba, W. "Learning Dexterous In-Hand Manipulation". In: *The International Journal of Robotics Research* 39.1 (2020), pp. 3–20. DOI: 10.1177/0278364919887447.

[21] Petrenko, A., Allshire, A., State, G., Handa, A., and Makoviychuk, V. "DexPBT: Scaling up Dexterous Manipulation for Hand-Arm Systems with Population Based Training". In: (May 2023).

[22] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. "Human-level control through deep reinforcement learning". In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236.

[23] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World". In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.

[24] Peng, X. B., Andrychowicz, M., Zaremba, W., and Abbeel, P. "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization". In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 3803–3810. DOI: 10.1109/ICRA.2018.8460528.

[25] OpenAI, Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., Petron, A., Paino, A., Plappert, M., Powell, G., Ribas, R., Schneider, J., Tezak, N., Tworek, J., Welinder, P., Weng, L., Yuan, Q., Zaremba, W., and Zhang, L. "Solving Rubik's Cube with a Robot Hand". In: *The International Journal of Robotics Research* 39.1 (2020), pp. 34–65. DOI: 10.1177/0278364919887447.

[26] Hinton, G., Vinyals, O., and Dean, J. "Distilling the Knowledge in a Neural Network". In: (Mar. 2015).

[27] Pomerleau, D. A. "ALVINN: An Autonomous Land Vehicle in a Neural Network". In: *Advances in Neural Information Processing Systems* 1 (1988).

[28] Levine, S., Finn, C., Darrell, T., and Abbeel, P. "End-to-End Training of Deep Visuomotor Policies". In: *Journal of Machine Learning Research* 17.39 (2016), pp. 1–40.

[29] Huang, D., Zhang, T., Li, Y., Zhao, L., Li, J., Fang, Z., Xia, C., Li, L., and He, X. "Dexterous Hand Manipulation via Efficient Imitation-Bootstrapped Online Reinforcement Learning". In: (Mar. 2025).

[30] Kumar, V., Todorov, E., and Levine, S. "Optimal control with learned local models: Application to dexterous manipulation". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2016, pp. 378–383. ISBN: 978-1-4673-8026-3. DOI: 10.1109/ICRA.2016.7487156.

[31] Makoviychuk, V., Wawrzyniak, L., Guo, Y., Lu, M., Storey, K., Macklin, M., Hoeller, D., Rudin, N., Allshire, A., Handa, A., and State, G. "Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning". In: *Thirty-fifth Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. 2021.

[32] Mittal, M., Yu, C., Yu, Q., Liu, J., Rudin, N., Hoeller, D., Yuan, J. L., Singh, R., Guo, Y., Mazhar, H., Mandlekar, A., Babich, B., State, G., Hutter, M., and Garg, A. "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments". In: *IEEE Robotics and Automation Letters* 8.6 (2023), pp. 3740–3747. DOI: 10.1109/LRA.2023.3270034.